

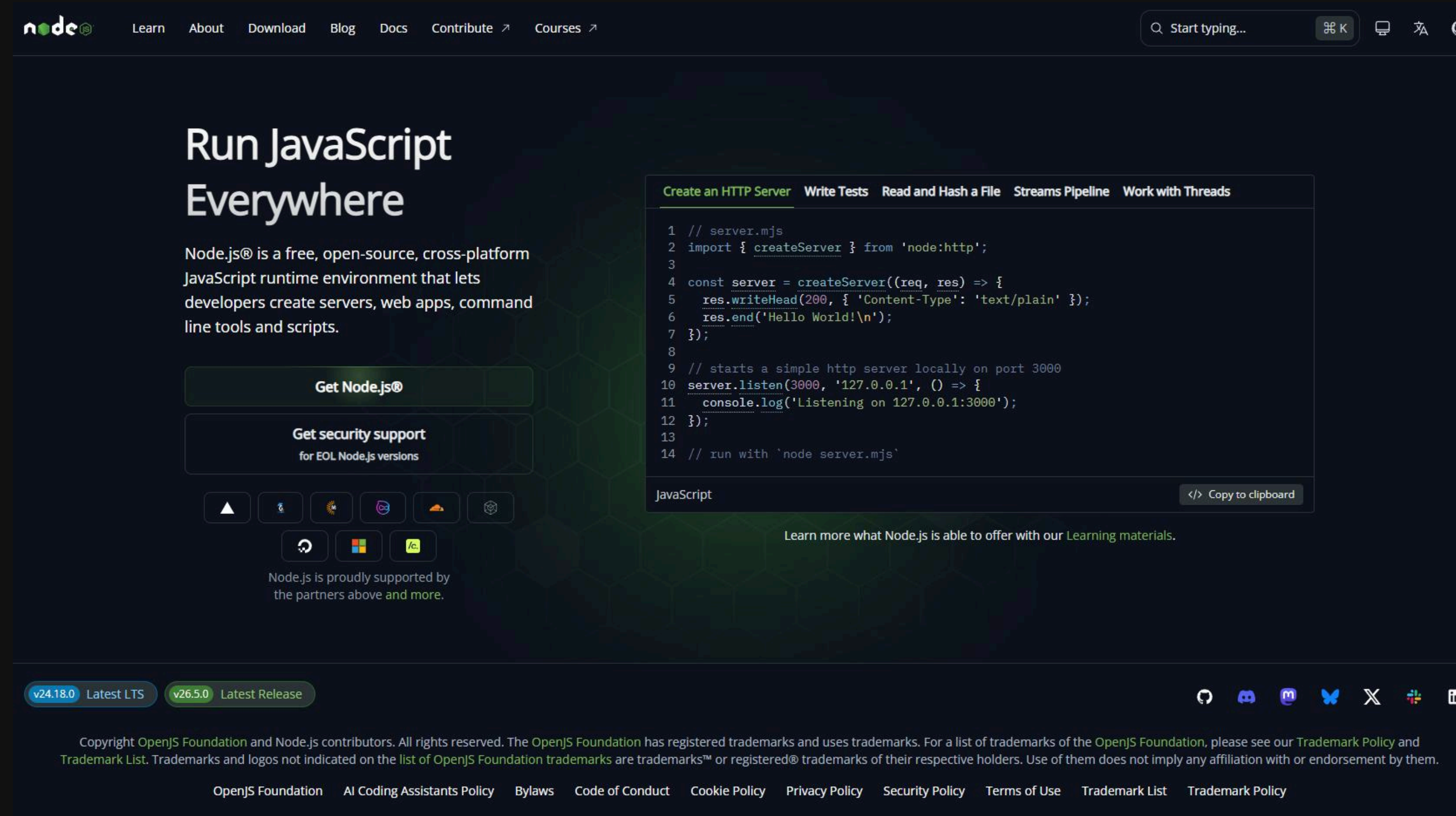


Server-Side Web Development (06016418)



วันนี้เราจะได้เรียนอะไรกันบ้าง ?





The screenshot shows the Node.js website homepage. The header includes the Node.js logo, navigation links (Learn, About, Download, Blog, Docs, Contribute, Courses), a search bar, and utility icons. The main content area features the heading "Run JavaScript Everywhere" and a description of Node.js as a free, open-source, cross-platform JavaScript runtime environment. Below this are two buttons: "Get Node.js®" and "Get security support for EOL Node.js versions". A row of partner logos is displayed, followed by the text "Node.js is proudly supported by the partners above and more." To the right, there is a code editor showing a JavaScript snippet for creating an HTTP server. The footer includes version information (v24.18.0 Latest LTS, v26.5.0 Latest Release), social media links, and a copyright notice for the OpenJS Foundation.

nodejs

Learn About Download Blog Docs Contribute Courses

Start typing...

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Get Node.js®

Get security support for EOL Node.js versions

Node.js is proudly supported by the partners above and more.

```
1 // server.mjs
2 import { createServer } from 'node:http';
3
4 const server = createServer((req, res) => {
5   res.writeHead(200, { 'Content-Type': 'text/plain' });
6   res.end('Hello World!\n');
7 });
8
9 // starts a simple http server locally on port 3000
10 server.listen(3000, '127.0.0.1', () => {
11   console.log('Listening on 127.0.0.1:3000');
12 });
13
14 // run with `node server.mjs`
```

JavaScript

Learn more what Node.js is able to offer with our Learning materials.

v24.18.0 Latest LTS v26.5.0 Latest Release

Copyright OpenJS Foundation and Node.js contributors. All rights reserved. The OpenJS Foundation has registered trademarks and uses trademarks. For a list of trademarks of the OpenJS Foundation, please see our Trademark Policy and Trademark List. Trademarks and logos not indicated on the list of OpenJS Foundation trademarks are trademarks™ or registered® trademarks of their respective holders. Use of them does not imply any affiliation with or endorsement by them.

OpenJS Foundation AI Coding Assistants Policy Bylaws Code of Conduct Cookie Policy Privacy Policy Security Policy Terms of Use Trademark List Trademark Policy

<https://nodejs.org/en>



พื้นฐาน JavaScript สำหรับ Node.js





Node.js คืออะไร ?

คือ เครื่องมือหรือ Runtime Environment ที่ช่วยให้เราสามารถเขียนภาษา JavaScript ฝั่ง Backend (ฝั่งเซิร์ฟเวอร์) ได้ จากเดิมที่ JavaScript มักจะทำงานแค่บน Frontend หรือหน้าเว็บเบราว์เซอร์เท่านั้น



เหตุผลที่ Node.js ได้รับความนิยม

- ใช้ภาษาเดียวทั้งระบบ: นักพัฒนาสามารถใช้ JavaScript เขียนทั้งหน้าบ้านและหลังบ้านได้เลย
- ประสิทธิภาพสูง: ออกแบบมาเพื่อรองรับงานจำนวนมากพร้อมกันด้วยสถาปัตยกรรมแบบ Event-driven และ Non-blocking I/O ทำให้ทำงานได้รวดเร็ว
- Community ขนาดใหญ่: มีผู้ใช้งานทั่วโลก ทำให้มี Library และเครื่องมือต่างๆ มากมายให้เลือกใช้
- รองรับ Real-time: เหมาะสำหรับการสร้างแอปพลิเคชันที่ต้องการอัปเดตข้อมูลแบบทันที เช่น แชท หรือ Live Dashboard

Asynchronous Programming



Callback Functions

คือฟังก์ชันที่ถูกส่งเข้าไปเป็น Argument
ให้กับอีกฟังก์ชันหนึ่ง เพื่อให้ฟังก์ชันหลักเรียกใช้งาน
(Execute) หลังจาก que การทำงานบางอย่างเสร็จสิ้น

จุดประสงค์หลัก :

- ใช้จัดการกับกระบวนการที่ทำงานแบบ Asynchronous (ไม่พร้อมกัน) ใน JavaScript
- ช่วยให้โปรแกรมสามารถทำงานอื่นต่อไปได้โดยไม่ต้องรอให้ งานที่ใช้เวลานาน (เช่น การดึงข้อมูลจาก Server หรือการอ่านไฟล์) ทำงานเสร็จก่อน

```
// ฟังก์ชันที่จำลองการทำงานแบบ Asynchronous
function fetchData(callback) {
    setTimeout(() => {
        const data = "Data received from the server.";
        callback(data);
    }, 2000); // จำลองการรอ 2 วินาที
}

console.log("Retrieving data...");

fetchData((data) => {
    console.log("Data received :", data);
});

console.log("The program continues running without waiting...");
```

<https://goonlinetools.com/snapshot/code/#f5uacti8o4gvf12bqyv84>



ข้อเสีย Callback Functions

Callback Hell คือปัญหาที่เกิดขึ้นเมื่อเราใช้ Callback Function ซ้อนกันหลายๆ ชั้นในโค้ดแบบ Asynchronous

ทำไมถึงเป็นปัญหา :

- อ่านยาก : โค้ดจะซ้อนกันเป็นรูปพีระมิด หรือที่มักเรียกกันว่า Pyramid of Doom ทำให้การไล่ลำดับการทำงานของโปรแกรมทำได้ลำบาก
- ดูแลรักษายาก : เมื่อโค้ดซับซ้อนขึ้น การแก้ไขบั๊กหรือการปรับปรุงโค้ดจะทำได้ยากมาก

Promise

Promise คือ Object

ที่ถูกสร้างขึ้นมาเพื่อใช้เป็นตัวแทนของผลลัพธ์ที่อาจจะเกิดขึ้นในอนาคต (โดยเฉพาะงานแบบ Asynchronous) ซึ่งช่วยแก้ปัญหา Callback Hell ทำให้โค้ดมีโครงสร้างที่อ่านง่ายและจัดการได้ดียิ่งขึ้น

การใช้งาน :

- ใช้ .then() เพื่อจัดการกับผลลัพธ์เมื่อ Promise ทำงานสำเร็จ
- ใช้ .catch() เพื่อจัดการกับข้อผิดพลาดหรือเมื่อ Promise ล้มเหลว

```
function fetchDataPromise() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const succes = true // ลองเปลี่ยนเป็น false เพื่อดูผลลัพธ์ของ reject
      if(succes) {
        const data = "Data received from the server.";
        resolve(data);
      } else {
        reject("Unable to retrieve data.");
      }
    }, 2000);
  });
}

console.log("Retrieving data...");

fetchDataPromise()
  .then((data) => {
    console.log("Data received :", data);
  })
  .catch((error) => {
    console.log("An error occurred :", error);
  });

console.log("The program continues running without waiting...");
```

<https://goonlinetools.com/snapshot/code/#4vdc04fczobrf7s23qukai>



Async / Await

คือ Syntax ที่ทันสมัยซึ่งสร้างขึ้นบนพื้นฐานของ Promise เพื่อให้การเขียนโค้ดแบบ Asynchronous (การทำงานที่ไม่รอให้เสร็จทีละขั้นตอน) มีรูปแบบที่สะอาดและอ่านง่ายเหมือนกับการเขียนโค้ดแบบปกติ (Synchronous)

หัวใจสำคัญของการทำงาน :

- `async` : ใช้เติมหน้าฟังก์ชันเพื่อระบุว่าเป็นฟังก์ชันที่ทำงานแบบ Asynchronous ซึ่งจะส่งค่ากลับเป็น Promise เสมอ
- `await` : ใช้ภายในฟังก์ชัน `async` เพื่อสั่งให้โปรแกรม รอ ให้ Promise ทำงานจนสำเร็จก่อนจึงจะรันโค้ดบรรทัดถัดไป
- การจัดการข้อผิดพลาด : แทนที่จะใช้ `.catch()` เหมือนใน Promise ทั่วไป เราจะใช้โครงสร้าง `try...catch` block ในการดักจับ Error แทน ซึ่งช่วยให้โค้ดดูเป็นระเบียบมากขึ้น



Async / Await

ข้อดี :

ช่วยลดความซับซ้อนของโค้ด ทำให้ไม่ต้องเขียน .then()

ซ้อนกันหลายชั้น

และทำให้โครงสร้างของโปรแกรมเข้าใจได้ง่ายขึ้นมากในงานที่ต้อง

ติดต่อกับฐานข้อมูลหรือเรียกใช้ API

```
function fetchDataPromise() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const succes = true // ลองเปลี่ยนเป็น false เพื่อดูผลลัพธ์ของ reject
      if (succes) {
        const data = "Data received from the server.";
        resolve(data);
      } else {
        reject("Unable to retrieve data.");
      }
    }, 2000);
  });
}

async function getAndPrintData() {
  console.log("Retrieving data...");
  try {
    const data = await fetchDataPromise();
    console.log("Data received :", data);
  } catch (error) {
    console.error("An error occurred :", error);
  }
}

getAndPrintData();

console.log("The program continues running without waiting...");
```

<https://goonlinetools.com/snapshot/code/#kyzo2pobb6kti9tlz6qw9>



เขียนโค้ด NodeJS และเข้าใจ Node Module System



CommonJS (CJS)

- ระบบดั้งเดิม : เป็นระบบ Module หลักของ Node.js ที่นิยมใช้กันมานาน
- วิธีใช้ : ใช้ `module.exports` ในการส่งออกฟังก์ชันหรือตัวแปร และใช้คำสั่ง `require()` ในการนำเข้า
- การทำงาน : ทำงานแบบ Synchronous (โหลดเสร็จก่อนถึงจะรันโค้ดบรรทัดถัดไป) ซึ่งเหมาะมากสำหรับการทำงานฝั่ง Server



CommonJS (CJS)



```
function add(a, b) {  
    return a+b;  
}  
  
module.exports = { add };
```

calculator.js



```
// Comon JS  
const calculator = require('./calculator')  
  
console.log(calculator.add(5, 3));
```

app.js



ES Modules (ESM)

- มาตรฐานใหม่ : เป็นมาตรฐานอย่างเป็นทางการของ JavaScript ตั้งแต่เวอร์ชัน ES2015 เป็นต้นมา
- วิธีใช้ : ใช้ export และ import ในการจัดการ
- การทำงาน : ทำงานแบบ Asynchronous (โหลดแบบไม่พร้อมกัน)
ทำให้มีประสิทธิภาพในการโหลดโค้ดที่ดีกว่า และสามารถใช้งานได้ทั้งใน Node.js และ Web Browser



ES Modules (ESM)



```
export function add(a, b) {  
    return a+b;  
}  
  
export const PI = 3.14;
```

calculator.js



```
// ES Modules  
import { add, PI } from './calculator.js'  
  
console.log(add(5, 3));  
console.log(PI);
```

app.js



CommonJS (CJS) VS ES Modules (ESM)

ข้อแตกต่างสำคัญ :

ไวยากรณ์ : CommonJS ใช้ `require()` แต่ ES Modules ใช้ `import`

นามสกุลไฟล์ : การใช้ ES Modules ใน Node.js มักต้องระบุนามสกุลไฟล์ `.js` ในคำสั่ง `import` ให้ชัดเจน



fs (File System)

fs (File System)

เป็นหนึ่งใน Core Modules ที่สำคัญที่สุดซึ่งมาพร้อมกับ Node.js โดยไม่ต้องติดตั้งเพิ่มเติม

```
// Core Modules
// fs (File System Module)

const fs = require('fs');

try {
  const data = fs.readFileSync('hello.txt', 'utf8');
  console.log(data);
} catch(err) {
  console.error(err);
}
```

<https://goonlinetools.com/snapshot/code/#cgwttb4e2ymbcx55ffv27i> (CJS)

```
// Core Modules
// fs (File System Module)

const { log } = require('console');
const fs = require('fs');

const content = 'Hello, NodeJS'

fs.writeFile('greeting.txt', content, err => {
  if (err) {
    console.error(err);
    return;
  }

  console.log('File written successfully.');
```

<https://goonlinetools.com/snapshot/code/#b2459nfz9es0r1xku2vi2jm> (ESM)



fs (File System)

หน้าที่หลักของ fs module :

- ใช้สำหรับจัดการไฟล์และโฟลเดอร์ในระบบคอมพิวเตอร์ของเราโดยตรง เช่น :
 - อ่านไฟล์ (Reading files) : ดึงข้อมูลจากไฟล์มาใช้งานในโปรแกรม
 - เขียนไฟล์ (Writing files) : สร้างไฟล์ใหม่หรือเขียนข้อมูลทับลงในไฟล์เดิม



path



path

path เป็นหนึ่งใน Core Modules ที่สำคัญซึ่งมีไว้เพื่อช่วยในการจัดการและทำงานกับ เส้นทางไฟล์ (file paths) และ โดเรกทอรี (directories) ภายในระบบปฏิบัติการ

หน้าที่หลักของ path module :

- จัดการความแตกต่างของ OS : ช่วยให้โปรแกรมของคุณทำงานได้ถูกต้องไม่ว่าจะเป็น Windows, macOS หรือ Linux โดยที่ path จะคำนึงถึงความแตกต่างของรูปแบบเส้นทาง (เช่น การใช้ / หรือ \) ให้โดยอัตโนมัติ
- สร้างเส้นทางไฟล์ : ใช้ฟังก์ชันอย่าง path.join() เพื่อเชื่อมต่อส่วนประกอบต่างๆ ของเส้นทางเข้าด้วยกันอย่างปลอดภัย



path



```
// Core Modules
```

```
// path
```

```
const path = require('path')
```

```
const filePath = path.join('users', 'john', 'documents', 'report.pdf')
```

```
console.log(filePath);
```

<https://goonlinetools.com/snapshot/code/#ve2hrwl97if537xqgekw>



os (Operating System)



os (Operating System)

os (Operating System) เป็นหนึ่งใน Core Modules ที่สำคัญ ซึ่งมาพร้อมกับ Node.js โดยไม่ต้องติดตั้งเพิ่มเติม

หน้าที่หลักของ os module : ใช้สำหรับเรียกดูข้อมูลและโต้ตอบกับ ระบบปฏิบัติการของเครื่องคอมพิวเตอร์ที่กำลังรัน Node.js อยู่ ช่วยให้โปรแกรมสามารถเข้าถึงทรัพยากรของเครื่องได้ เช่น :

- Platform : ตรวจสอบว่ารันอยู่บนระบบปฏิบัติการอะไร (เช่น Windows, macOS หรือ Linux)
- Memory : ดูข้อมูลหน่วยความจำ (RAM) ของเครื่อง
- CPU : ดูรายละเอียดเกี่ยวกับหน่วยประมวลผล
- System Uptime : ดูระยะเวลาที่เปิดเครื่องมาแล้ว



os (Operating System)

```

// Core Modules
// os (Operating System)

const os = require('os');

console.log(`OS Platform : ${os.platform()}`);
console.log(`Total Memory : ${os.totalmem()} bytes`);
console.log(`Free Memory : ${os.freemem()} bytes`);
console.log(`Uptime : ${os.uptime()} seconds`);
```

<https://goonlinetools.com/snapshot/code/#vw3kvjvhqbb8yru77ko6t4>





npm (Node Package Manager)

npm (Node Package Manager) คือเครื่องมือจัดการแพ็คเกจ (Package) ที่สำคัญที่สุดสำหรับโปรเจกต์ Node.js โดยมีหน้าที่หลักคือช่วยให้เราสามารถดาวน์โหลด ติดตั้ง และจัดการ Library หรือเครื่องมือต่างๆ จากชุมชนนักพัฒนาทั่วโลกมาใช้งานในโปรเจกต์ของเราได้ง่ายขึ้น



npm (Node Package Manager)

- `npm init` : ใช้สำหรับเริ่มต้นโปรเจกต์ใหม่ ซึ่งจะสร้างไฟล์ `package.json` ขึ้นมา
- `npm init -y` : เป็นคำสั่งลัดในการสร้างไฟล์ `package.json` แบบอัตโนมัติโดยไม่ต้องตอบคำถามทีละข้อ
- `npm install` (หรือ `npm i`) : ใช้สำหรับติดตั้ง Library ที่จำเป็นต่อโปรเจกต์ โดยจะบันทึกรายชื่อไว้ใน `dependencies` ของไฟล์ `package.json`
- `npm install -D` : ใช้สำหรับติดตั้ง Library ที่จำเป็นเฉพาะในช่วง การพัฒนา (Development) เท่านั้น เช่น `nodemon`
- `npm uninstall` : ใช้สำหรับลบแพ็คเกจที่ไม่ต้องการออกจากโปรเจกต์



package.json

ความสำคัญของไฟล์ package.json :

ไฟล์นี้เปรียบเสมือน "บัตรประชาชน" ของโปรเจกต์ ซึ่งจะทำหน้าที่เก็บข้อมูลสำคัญต่างๆ เช่น ชื่อโปรเจกต์, เวอร์ชัน, และรายชื่อของ Dependency ทั้งหมดที่โปรเจกต์ต้องใช้ เพื่อให้คนอื่นสามารถนำโค้ดของเราไปรันต่อได้ง่ายเพียงแค่รันคำสั่งติดตั้งแพ็คเกจ



package.json

โดยมีองค์ประกอบหลักที่สำคัญดังนี้ :

- name : ชื่อของโปรเจกต์ (ต้องเป็นตัวพิมพ์เล็ก)
- version : เลขเวอร์ชันของโปรเจกต์ (เช่น 1.0.0)
- description : คำอธิบายสั้นๆ เกี่ยวกับโปรเจกต์
- main : ไฟล์เริ่มต้น (entry point) ของโปรเจกต์ (โดยทั่วไปคือ app.js หรือ index.js)
- scripts : ส่วนสำหรับกำหนดคำสั่งรันโปรเจกต์ เช่น คำสั่งเริ่มต้นหรือคำสั่งเทสต์ (เช่น npm run dev หรือ npm run start)
- dependencies : รายชื่อ package หรือ library ภายนอกที่จำเป็นสำหรับการทำงานจริงของโปรเจกต์
- devDependencies : รายชื่อ package ที่จำเป็นเฉพาะในช่วงการพัฒนา (Development) เท่านั้น เช่น nodemon ซึ่งช่วยในการรีสตาร์ทเซิร์ฟเวอร์อัตโนมัติเมื่อมีการแก้ไขโค้ด
- keywords : คำค้นหาที่เกี่ยวข้องกับโปรเจกต์เพื่อช่วยให้คนอื่นค้นหาเจอได้ง่ายขึ้น
- author / license : ข้อมูลผู้เขียนและประเภทลิขสิทธิ์ของโค้ด



สร้างและรัน script ใน package.json



```
{
  "name": "day04",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "devDependencies": {
    "nodemon": "^3.1.14"
  },
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

<https://goonlinetools.com/snapshot/code/#dbt6wdo2tbaodx2mr851eh>



สร้าง Server ตัวแรกด้วย NodeJS



Setup Server

```
const http = require('http');
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');

  res.end('Hello, World!');
});

server.listen(port, () => {
  console.log('Server running...');
});
```

<https://goonlinetools.com/snapshot/code/#27cyycethwdge4id8tt7tf>



HTTP STATUS CODE CHEAT SHEET

รวมเด็ด ! ได้สถานะของ HTTP ที่ทุกคนต้องรู้ !

1XX Informational การแจ้งข้อมูลหรือสถานะ

100 Continue
ได้รับ Request ส่วนแรกแล้ว แต่ Server ยังไม่ทำการ Response กลับมา รอให้ Request จากทาง Client มาครบก่อนจึงจะตอบกลับ

101 Switching Protocols
Server ทราบถึงการอยากเปลี่ยน Protocol ของ Client โดย Server เป็นคนบอกว่ามี Protocol อะไรที่รองรับให้เปลี่ยนได้บ้าง

102 Processing
เป็นการตอบกลับมาแบบชั่วคราวว่ากำลังดำเนินการอยู่ แต่ยังไม่ประมวลผลสำเร็จ

2XX Success การแจ้งเมื่อดำเนินการสำเร็จ

200 OK
ทุกอย่างเรียบร้อยและส่ง HTML Response กลับมา หลังจากการประมวลผลนั้นๆเรียบร้อยแล้ว

201 Created
Request ที่ร้องขอ ถูกสร้างขึ้นในตำแหน่งที่กำหนดสำเร็จ

202 Accepted
ทางฝั่ง Server ได้รับ Request ที่ส่งมาแล้ว แต่ Server ยังคงประมวลผลอยู่

203 Non-authoritative Information
Request ที่ส่งมาได้รับการประมวลผลแล้ว แต่ Payload ที่ส่งกลับมากถูกแก้ไขโดย Proxy ที่อยู่กึ่งกลางจาก Server จริงๆ

204 No Content
Server ประมวลผลเรียบร้อยแล้ว แต่ Response ที่จะตอบกลับไปในนั้นไม่มี Response Body

205 Reset Content
Server ได้รับ Request แล้วและต้องการให้เปลี่ยนเป็น Document View ซึ่งทำให้ User ต้องส่ง Request มาใหม่

206 Partial Content
Request ได้รับแล้ว โดยจะส่งข้อมูลให้บางส่วนตามที่ Client ต้องการระบุไว้ใน Range Header

207 Multi-Status
เป็นการตอบกลับที่ภายในมี Information ที่หลากหลายจากหลาย ๆ Resource ในสถานะการที่มี Status Code ที่หลากหลาย

3XX Redirection การแจ้งเปลี่ยนเส้นทาง

300 Multiple Choices
Response มีการ Redirect มากกว่า 1 URL แต่ไม่เกิน 5 URL แล้วให้ทาง Client เลือก URL

301 Moved Permanently
เป็น Response ที่จะย้าย URL ที่เราเรียกไปยังตำแหน่งนั้น ๆ อย่างถาวร

302 Found
เป็น Response บอกว่า URL ที่ถูกเรียกมีการย้ายไป URL อื่นชั่วคราว

303 See Other
เป็น Response ที่ย้าย User ให้ไปที่ Resource อื่น

304 Not Modified
เป็น Response ของที่บอกว่า Request GET หรือ HEAD ที่ส่งมานั้นไม่มีการเปลี่ยนแปลงจากครั้งสุดท้าย

4XX Client Error ข้อผิดพลาดฝั่ง Client

400 Bad Request
บอก Client ว่า Syntax, Request Body ไม่ถูกต้อง

401 Unauthorized
บอก Client ว่ายังไม่ได้รับยืนยันตัวตนก่อนส่ง Request

402 Payment Required
เป็น Feature ที่ใช้ในอนาคต เพื่อร้องขอการจ่ายเงิน

403 Forbidden
บอก Client ว่าไม่มีสิทธิ์เข้าถึง Resource ดังกล่าว

404 Not Found
บอก Client ว่าไม่พบหน้าที่ Client ร้องขอเข้ามา

405 Method Not Allowed Method
ไม่อนุญาตให้ใช้ Method นั้นใน Resource ดังกล่าว

406 Not Acceptable
บอก Client ว่าการเรียกใช้งานไม่เข้ากับ Request

408 Request Time Out
ใช้เวลานานในการ Request เกินกว่าที่ Server กำหนด

413 Request Entity Too Large
Request ที่ส่งเข้ามาใหญ่เกินไป

414 Request-URL Too Long
URL ที่ส่งเข้ามายาวเกินไป

415 Unsupported Media Type
Server ไม่ Support ชนิดของสื่อที่ Client ร้องขอ

5XX Server Error ข้อผิดพลาดฝั่ง Server

500 Bad Request
เกิดข้อผิดพลาดทางฝั่ง server แบบไม่ทราบสาเหตุ

501 Unauthorized
Server ไม่รองรับ Request นั้นๆ

502 Payment Required
Server ไม่สามารถติดต่อไปยัง Engine หรือ Proxy ที่ใช้สำหรับการประมวลผล

503 Service Unavailable
Server ไม่พร้อมให้บริการ

504 Gateway Timeout
Server ไม่ได้รับ Response จากฝั่งของ Engine หรือ Server อื่น ๆ ในเวลาที่กำหนด

หากต้องการดู response code ทั้งหมด แนะนำให้เข้าไปดูที่ <https://httpstatuses.com>



มารู้จักโครงสร้างของ HTTP MESSAGES

มีรายละเอียดยังไง ติดตามได้ที่... **กันเถอะ!**

HTTP Messages เป็นวิธีการแลกเปลี่ยนข้อมูล
ระหว่าง **Server** และ **Client**



HTTP Request
คำขอที่ส่งโดย Client
เพื่อดำเนินการบน Server



HTTP Response
การตอบกลับจาก Server
ที่ส่งไปยัง Client

GET /test/hi-there.txt HTTP/1.0

Accept: text/*
Accept-Language: en,fr

Start line

Headers

Body

HTTP/1.0 200 OK

Content-type: text/plain
Content-length: 19

Hi ! I'm a message !

โครงสร้างของ HTTP MESSAGES

Start-line
ใช้ระบุว่าการทำอะไรกับ Request และ Response

- Request Message**

HTTP Method - อธิบาย Action ภายใน Messages
เช่น GET, PUT, หรือ POST

URL - เป็นพอร์ตหรือเส้นทางเป้าหมาย

HTTP version - เวอร์ชันของ HTTP นั้นเอง
- Response Message**

Protocol Version - เช่น HTTP/1.1

Status Code - ระบุว่าคำขอสำเร็จหรือล้มเหลว จะแทน
ด้วยหมายเลขต่าง ๆ เช่น 404, 200 หรือ 302

Status Text - ข้อความระบุสถานะ เช่น Not Found

HTTP Headers
ประกอบด้วย
Name - Value คั่นด้วย
เครื่องหมาย colon (:)
ใช้เพื่ออธิบายเนื้อหาใน
Messages

Body
เนื้อหาเพิ่มเติมที่สำคัญ
จะถูกระบุโดย Start-line
และ HTTP
Headers





การทำ Routing เบื้องต้น



```
const http = require('http');
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');

  if (req.url === '/') {
    res.end('Home');
  } else if (req.url === '/about') {
    res.end('About Me');
  } else if (req.url === '/contact') {
    res.end('Contact');
  } else {
    res.statusCode = 404;
    res.end('Not Found');
  }
});

server.listen(port, () => {
  console.log('Server running...');
});
```

<https://goonlinetools.com/snapshot/code/#x9mimz6i8pna1852kop7jc>



การส่งข้อมูลกลับ (Response)



```
const http = require('http');
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'application/json');

  const data = {
    message: 'Hellom World',
    data: {
      name: 'NodeJS Server',
      version: '1.0'
    }
  }

  res.end(JSON.stringify(data));
});

server.listen(port, () => {
  console.log('Server running...');
});
```

<https://goonlinetools.com/snapshot/code/#t91bdte3m3sh5p8hk30an>



การจัดการ Request สำหรับอ่านข้อมูลจาก URL และ Headers



URL



```
const http = require('http');
const port = 3000;
const url = require('url');

const server = http.createServer((req, res) => {

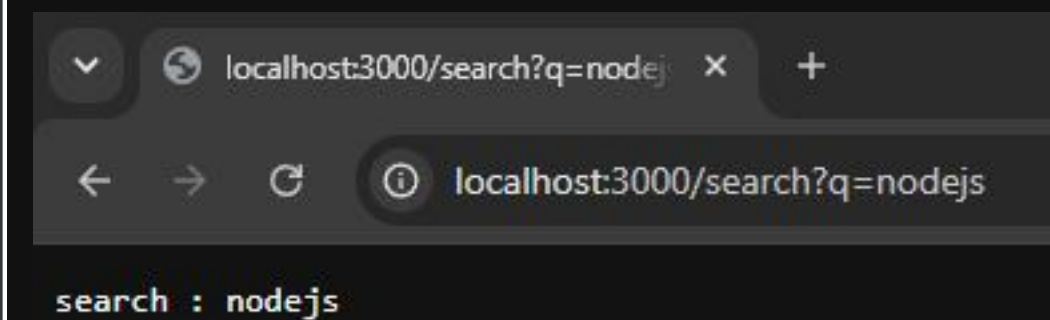
  // Read data from URL
  // Path : /users/1
  // Query String : /search?q=nodejs

  const parsedUrl = url.parse(req.url, true);
  const path = parsedUrl.pathname;
  const query = parsedUrl.query;

  if (path === '/search') {
    res.statusCode = 200
    res.setHeader('Content-Type', 'text/plain; charset=utf-8')
    const searchTerm = query.q || 'Search term not specified'
    res.end(`search : ${searchTerm}`)
  } else {
    res.statusCode = 404
    res.end('Not Found')
  }

});

server.listen(port, () => {
  console.log('Server running...');
});
```



<https://goonlinetools.com/snapshot/code/#tqre3or0hpp20z7d9sdgd>



Headers



```
const http = require('http');
const port = 3000;
const url = require('url');

const server = http.createServer((req, res) => {

  res.statusCode = 200
  res.setHeader('Content-Type', 'text/plain; charset=utf-8')

  const userAgent = req.headers['user-agent'] || 'Not Found User Agent'
  res.end(`Your User-Agent is ${userAgent}`);

});

server.listen(port, () => {
  console.log('Server running...');
});
```

<https://goonlinetools.com/snapshot/code/#v0xyk1ug5k6d7dxtts5zu>



express



Express

Express.js ซึ่งเป็น Framework ยอดนิยมสำหรับ Node.js เพื่อช่วยให้การพัฒนาเว็บแอปพลิเคชันหรือ REST API มีประสิทธิภาพและรวดเร็วขึ้น

ข้อดีหลักของ Express.js

- พัฒนาได้รวดเร็ว : มีโครงสร้างที่เป็นมาตรฐาน ไม่ต้องเขียนโค้ดซ้ำซ้อน
- ระบบ Routing ที่มีประสิทธิภาพ : ช่วยจัดการเส้นทาง URL ได้ง่ายและเป็นระเบียบ
- Middleware ที่ยืดหยุ่น : เป็นหัวใจสำคัญที่ช่วยให้เราแทรกฟังก์ชันการทำงานต่างๆ (เช่น ตรวจสอบสิทธิ์ หรือ บันทึก Log) ในวงจรของ Request ได้
- ชุมชนขนาดใหญ่ : มีตัวอย่างโค้ดและไลบรารีเสริมมากมายให้ศึกษา
- ประหยัดทรัพยากร : ออกแบบมาให้มีขนาดเล็กและไม่ซับซ้อน



```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello, Express');
});

app.listen(port, () => {
  console.log('Server is running...');
});
```

<https://goonlinetools.com/snapshot/code/#ogv8nixotr9xmoyh9rqdbm>



การใช้งาน Express Router



```
const express = require('express');
const app = express();
const port = 3000;

const userRouter = require('./user');

app.use('/user', userRouter);

app.get('/', (req, res) => {
  res.send('Hello, Express');
});

app.listen(port, () => {
  console.log('Server is running...');
});
```

<https://goonlinetools.com/snapshot/code/#pjh9rp2ainre3zcaphb5d>

```
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => {
  res.send('User home page');
});

router.get('/:id', (req, res) => {
  res.send(`User ID: ${req.params.id}`);
});

module.exports = router;
```

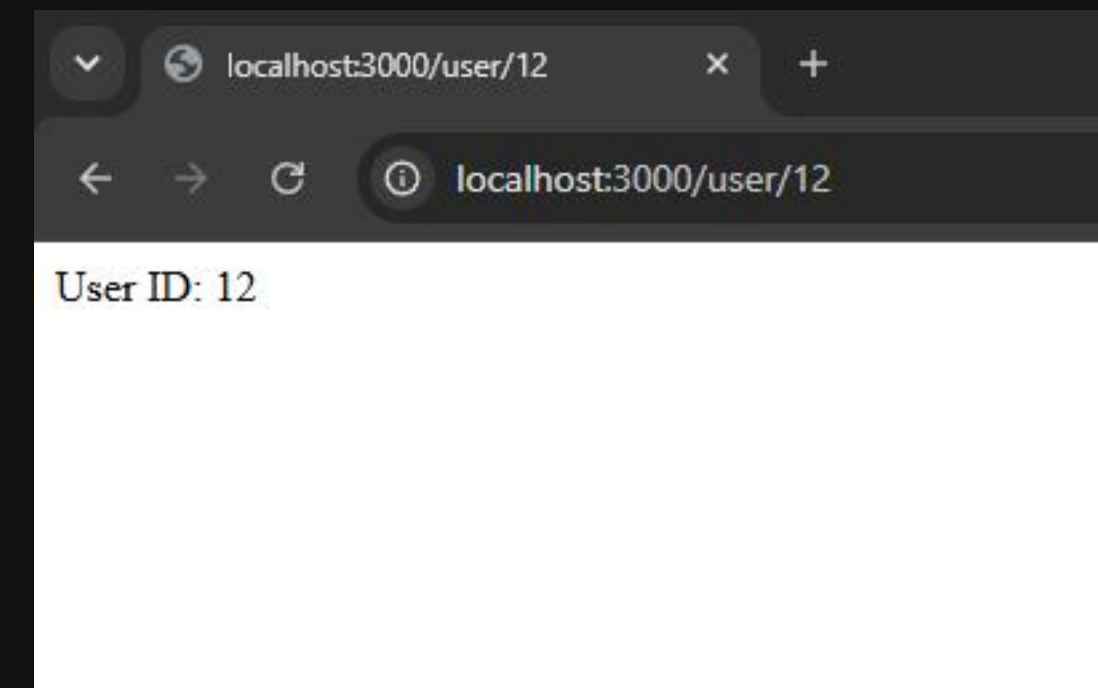
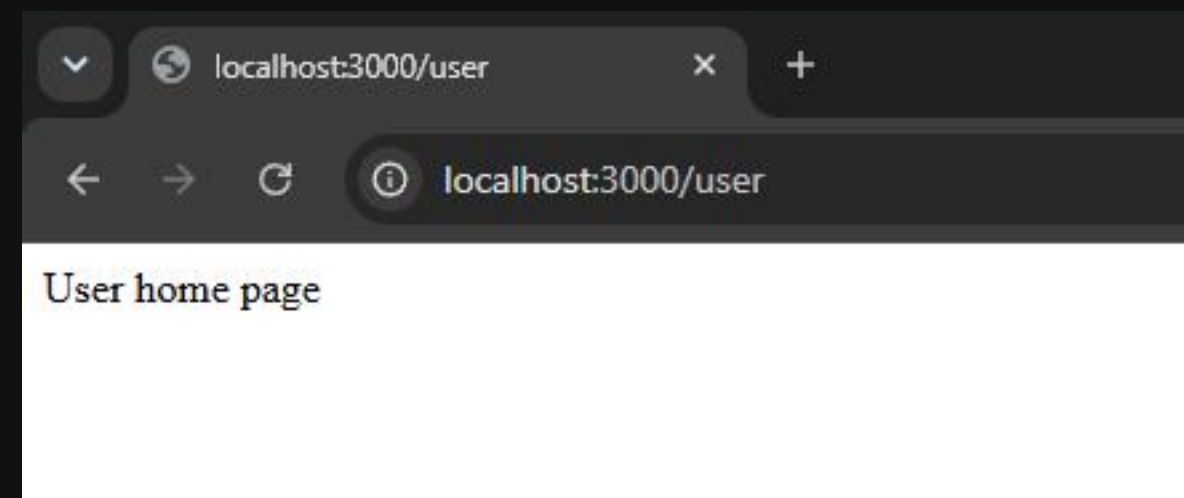
<https://goonlinetools.com/snapshot/code/#ovp36y9t0khv5wpr4ln7p>



Express Router

ประโยชน์และการใช้งาน:

- การจัดระเบียบโค้ด: แทนที่จะเขียนทุกเส้นทางไว้ในไฟล์หลัก (เช่น app.js) เราสามารถแยกออกมาเป็นไฟล์เฉพาะกิจ (เช่น routes/users.js) ตามฟังก์ชันการทำงานได้
- การนำกลับมาใช้ใหม่: โค้ดในส่วนของ Router สามารถนำไปใช้ซ้ำในส่วนอื่นๆ ของแอปพลิเคชันได้ง่าย





Quiz



<https://forms.gle/eCRAzTWbPHHmR3Bd6>